
System Design — Volume II

Advanced Distributed Systems | Senior Staff Level

Apple Staff / Principal Engineer Interview Preparation

System Design Series — Printer-Friendly Edition

System Design — Volume II

Apple Inc Senior Staff Engineer Interview Prep | Advanced Distributed Systems

■ = Frequently asked at Apple | ■ = Basic | ■ = Intermediate | ■ = Advanced

>

Volume I covers Staff Engineer-level topics: scalability fundamentals, core building blocks, and classic design problems. **Volume II** covers Senior Staff Engineer-level topics: distributed systems theory, advanced consistency patterns, and complex multi-system designs.

Table of Contents

Part A — Distributed Systems Theory

1. [CAP Theorem & PACELC](#chapter-1-cap-theorem--pacelc)
2. [Consistency Models — Strong, Eventual, Causal](#chapter-2-consistency-models)
3. [Consensus Algorithms — Raft & Paxos](#chapter-3-consensus-algorithms)
4. [Distributed Transactions — 2PC, Saga Pattern](#chapter-4-distributed-transactions)

Part B — Advanced Data Patterns

5. [CQRS — Command Query Responsibility Segregation](#chapter-5-cqrs)
6. [Event Sourcing](#chapter-6-event-sourcing)
7. [Data Pipelines — Lambda vs Kappa Architecture](#chapter-7-data-pipelines)

Part C — Reliability & Observability

8. [Observability — Metrics, Tracing, Logging](#chapter-8-observability)
9. [Global Distribution & Multi-Region](#chapter-9-global-distribution--multi-region)
10. [Real-Time Systems — WebSockets, SSE, CRDT](#chapter-10-real-time-systems)

Part D — Senior-Level Design Problems

11. [Design Google Docs — Collaborative Editing](#chapter-11-design-google-docs)
 12. [Design a Distributed Cache — Redis Internals](#chapter-12-design-a-distributed-cache)
 13. [Design a Payment System](#chapter-13-design-a-payment-system)
 14. [Design Uber / Lyft — Real-Time Ride Matching](#chapter-14-design-uber--lyft)
 15. [Design a Distributed Search Engine](#chapter-15-design-a-distributed-search-engine)
-

Part A — Distributed Systems Theory

Chapter 1: CAP Theorem & PACELC

Q1 ■ ■ What is the CAP Theorem? Give real-world examples.

Plain English First

In any distributed system that has network partitions (and all real networks do), you must choose **at most two** of:

```
C – Consistency    Every read sees the most recent write (or an error)
A – Availability    Every request receives a response (no errors, no timeouts)
P – Partition Tol. System continues operating when network splits occur
```

Since **P is non-negotiable** in real distributed systems (network partitions will happen — cables get cut, routers fail), the real choice is:

```
CP – Consistent + Partition Tolerant
    During a partition: return error instead of stale data
    "I'd rather tell you I don't know than tell you something wrong"
    Examples: HBase, ZooKeeper, Etcd, Redis Cluster (by default)

AP – Available + Partition Tolerant
    During a partition: return possibly stale data
    "I'd rather give you my best guess than say I'm unavailable"
    Examples: Cassandra, DynamoDB, CouchDB, DNS
```

Real-World Trade-offs

Scenario: Bank balance
User's actual balance: \$100
Network partition happens
Replica still thinks balance is \$500 (old value)

CP system (HBase):
During partition: "Error: cannot verify balance"
User can't withdraw – frustrating but SAFE (no accidental overdraft)

AP system (Cassandra):
During partition: returns \$500 (stale)
User might withdraw money they don't have – DANGEROUS for banking

Scenario: Amazon shopping cart
Jeff Bezos's famous decision: shopping cart = AP
"It's better to show you a cart with an extra item (stale) than to show you an empty cart when our DB is partitioned"
Worst case: user removes item, it reappears → minor UX issue
vs: cart unavailable during checkout → lost sale

Apple Pay:
Payments = CP (never show wrong balance, better to fail than double-charge)
App Store recommendations = AP (showing yesterday's charts is fine)

Q2 ■ What is PACELC? Why does it extend CAP?

CAP is only about behavior during partitions (rare events).
PACELC captures the more common trade-off: normal operation.

PACELC:
If Partition: choose between Availability vs Consistency (same as CAP)
Else (normal): choose between Latency vs Consistency

PA/EL – Available during partition, Low latency normally
DynamoDB (default), Cassandra
"I'll take the fastest path and accept stale reads"

PC/EC – Consistent during partition, Consistent normally (slower)
HBase, ZooKeeper, traditional RDBMS
"Always strong consistency, I'll pay the latency cost"

PC/EL – Consistent during partition, Low latency normally
Impossible to achieve perfectly (lower latency = fewer sync points)

PA/EC – Available during partition, Consistent normally (less common)

Real meaning for engineers:
Even with no partition, getting consistency across replicas costs latency
(quorum reads, synchronous replication)
The trade-off is ALWAYS latency vs consistency, not just during failures

Chapter 2: Consistency Models

Q3 ■ ■ What are the different consistency models? When do you use each?



Linearizability (Strongest)

Every operation appears to take effect at a single instant in real time.
The system behaves as if there is one copy of the data.

Timeline:

t=1: Client A writes X=1
t=2: Client B reads X → MUST see 1 (not 0)
t=3: Client A reads X → MUST see 1

Cost: Every read must check all replicas (quorum read) or use a leader

Latency: High (must synchronize before responding)

Use for: Leader election, distributed locks, counters that must be exact

Systems: ZooKeeper, etcd, Spanner

Serializable (Database Transactions)

Transactions appear to execute serially (one at a time), in some order.
A transaction that begins AFTER another finishes will see its effects.

Stronger than eventual, but doesn't require real-time ordering
(a committed transaction from 5 minutes ago might not appear "now" in strict time)

Use for: Database ACID transactions (PostgreSQL default isolation)

Cost: Locking / MVCC overhead

Causal Consistency

Operations that are causally related appear in the right order to all processes.
Concurrent (causally unrelated) operations can be seen in any order.

Example:

Alice posts "I'm getting married!" (write W1)
Bob reads W1, then comments "Congratulations!" (write W2, caused by W1)
Carol must see W1 before W2 (causal order maintained)
But Carol might see Bob's comment before Alice's post if they're unrelated to Carol

Vector Clocks track causality:

Each process maintains a vector of logical timestamps
[Alice:3, Bob:2, Carol:1] means: "I've seen 3 of Alice's events, 2 of Bob's..."
If my vector dominates yours → I'm causally after you
If neither dominates → concurrent (no causal relationship)

Use for: Collaborative apps, comment threads, social feeds
Systems: DynamoDB (with causality tokens), MongoDB sessions

Read-Your-Writes

After a client performs a write, that same client always reads its own write.
Other clients might still see old data.

Example:

You update your profile picture
You immediately visit your profile → you MUST see your new picture
(other users might see the old one for a while – that's OK)

Implementation:

After write: remember the write timestamp T
For reads from same session: "read at timestamp $\geq T$ " → route to primary or
wait until replica catches up to T

Use for: User-facing write → read scenarios (profile updates, settings)

Eventual Consistency (Weakest)

Given no new writes, all replicas will converge to the same value eventually.
No guarantee on when. Reads may return stale data.

Example: DNS propagation

You update your domain's IP → propagates in 24-48 hours
Some DNS servers still serve old IP during propagation
Eventually all DNS servers agree

Use for: Anything where stale reads are acceptable (analytics, social counts)
Systems: Cassandra (default), DynamoDB (default), S3

Conflict resolution strategies:

Last Write Wins (LWW): highest timestamp wins (risk: clock skew causes data loss)
CRDT: data types that merge automatically (counters, sets, maps)
Application-defined merge: developer writes merge function

Chapter 3: Consensus Algorithms

Q4 ■ ■ What is the Raft consensus algorithm? Why does it matter for system design?

Plain English First

Raft is how distributed systems (Kafka, etcd, CockroachDB, TiKV) agree on a single truth when nodes can fail or messages can be lost. Think of it as **democratic voting among servers** to elect a leader and agree on every log entry.

Why consensus matters:

You have 3 database replicas. One gets a write. The others must agree the write happened – even if the leader crashes before telling them all.

Without consensus: split-brain (two nodes think they're leader, diverging data)

With consensus: only one leader at a time, all commits are durable

Raft simplified:

1. Leader Election

All nodes start as Followers

If Follower hears no heartbeat from Leader within timeout (150-300ms):
becomes Candidate, starts election

Candidate votes for itself, requests votes from others

First to get majority ($\lceil N/2 \rceil + 1$) wins, becomes new Leader

2. Log Replication

ALL writes go to the Leader

Leader appends to its log, sends AppendEntries to all Followers

When majority ACK: Leader commits, applies to state machine, responds to client

Followers eventually apply committed entries

3. Safety guarantee

A Leader is elected only if it has the most complete log

→ you never lose a committed entry even if leader crashes

Cluster of 5 nodes:

Can tolerate 2 failures ($5/2 = 2 \rightarrow$ need 3 for majority)

During failure: remaining 3 still form majority → elect new leader

With 3 nodes: tolerates 1 failure

Rule: deploy odd numbers (3, 5, 7) – even numbers don't improve fault tolerance

When to mention in interview:

"For leader election / distributed lock / configuration store,
I'd use etcd which uses Raft consensus"

"Kafka uses ZooKeeper (Zab protocol, similar to Paxos) for controller election"

"This gives us linearizable writes with $(N-1)/2$ fault tolerance"

Chapter 4: Distributed Transactions

Q5 ■ ■ What is Two-Phase Commit (2PC)? What are its problems?

Plain English First

2PC coordinates a transaction across multiple services/databases so that either **all** commit or **all** abort. Like a wedding officiant asking "does anyone object?" before pronouncing the couple married.

Participants: Order Service DB, Inventory DB, Payment DB

Phase 1 – Prepare (Voting)

Coordinator → Order DB: "Can you commit order 123?"
Coordinator → Inventory DB: "Can you reserve item 456?"
Coordinator → Payment DB: "Can you charge \$99.99?"

Each participant:

Acquires locks, does the work (but doesn't commit yet)
Writes to its local undo log (for rollback)
Responds: VOTE_COMMIT or VOTE_ABORT

Phase 2 – Commit or Abort

All voted COMMIT → Coordinator sends GLOBAL_COMMIT to all
Any voted ABORT → Coordinator sends GLOBAL_ABORT to all

Each participant: commits or rolls back, releases locks

```
■■■■■■■■■■■■■■■■■■■■ prepare ■■■■■■■■■■■■■■■■■■■
■ Coordinator ■ ■■■■■■■■■■■■■■■■■■■ Order DB ■ ■■■ VOTE_COMMIT
■ ■■■■■■■■■■■■■■■■■■■ Inventory DB ■ ■■■ VOTE_COMMIT
■ ■■■■■■■■■■■■■■■■■■■ Payment DB ■ ■■■ VOTE_COMMIT
■ ■■■■■■■■■■■■■■■■■■■ commit ■■■■■■■■■■■■■■■■■■■
■ ■■■■■■■■■■■■■■■■■■■ all three ■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■
```

Problems with 2PC

1. Blocking protocol
If coordinator crashes AFTER sending prepare but BEFORE sending commit:
Participants hold locks indefinitely, waiting for coordinator to recover
System is stuck (blocking)
2. Single point of failure
Coordinator crashes → all participants blocked
3. Locks held across network round-trips
Phase 1 + network RTT + Phase 2 = high latency
Locks held the entire time → low throughput
4. Not suited for microservices
Each service has its own DB (different tech)
2PC requires all DBs to support the same distributed transaction protocol
PostgreSQL ↔ MongoDB ↔ Redis → no common 2PC standard

When 2PC is acceptable:
Same technology across all participants (all PostgreSQL)
Short transactions (locks held briefly)
Low volume (not millions of TPS)
XA transactions in Java EE / JTA

Q6 ■ ■ What is the Saga Pattern? Choreography vs Orchestration?

Plain English First

The Saga Pattern replaces a single distributed transaction with a **sequence of local transactions**, each with a **compensating transaction** (undo action) if something goes wrong later.

Think of it like booking a vacation: you book the flight, hotel, and rental car separately. If the car rental fails, you cancel the hotel, then cancel the flight. Each step can be undone individually.

Order Saga:

Step 1: Create Order (Order Service)	→ compensate: Cancel Order
Step 2: Reserve Inventory (Inventory Service)	→ compensate: Release Inventory
Step 3: Charge Payment (Payment Service)	→ compensate: Refund Payment
Step 4: Notify User (Notification Service)	→ compensate: Send cancellation email

Happy path: Step 1 → Step 2 → Step 3 → Step 4 → DONE

Payment fails: Step 1 → Step 2 → Step 3 FAILS

Compensate: Step 2 undo (release inventory) → Step 1 undo (cancel order)

Choreography (Event-based, decentralized)

No central coordinator. Each service listens to events and reacts.

Order Service → publishes: OrderCreated



Inventory Svc → listens: OrderCreated → reserves stock → publishes: StockReserved



Payment Svc → listens: StockReserved → charges card → publishes: PaymentCharged



Notification → listens: PaymentCharged → sends email

If Payment fails:

Payment Svc → publishes: PaymentFailed



Inventory Svc → listens: PaymentFailed → releases stock → publishes: StockReleased



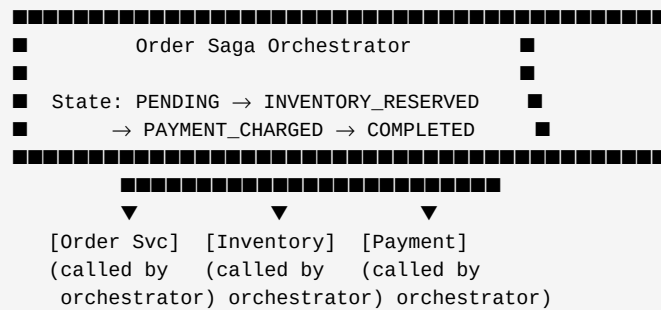
Order Svc → listens: StockReleased → cancels order

Pros: Loose coupling, no single point of failure, services are independent

Cons: Hard to track overall saga state, difficult to debug, risk of cyclic events

Orchestration (Central coordinator)

One Saga Orchestrator drives the entire flow. It knows the full state.



Orchestrator sends command → receives response → decides next step

If Payment fails:

- Orchestrator detects failure
- Calls Inventory Service: "compensate, release stock"
- Calls Order Service: "compensate, cancel order"
- Updates saga state to: FAILED

Pros: Clear state visibility, easy to debug, centralized retry logic

Cons: Orchestrator can become a bottleneck, single point of failure
(mitigate with persistent state in DB + idempotent steps)

Implementation: Store saga state in DB

```
CREATE TABLE order_sagas (  
  saga_id      UUID PRIMARY KEY,  
  order_id     BIGINT,  
  current_step VARCHAR(50),  -- 'INVENTORY_RESERVED', 'PAYMENT_CHARGED', etc.  
  status       VARCHAR(20),  -- 'IN_PROGRESS', 'COMPLETED', 'FAILED'  
  payload      JSONB,  
  created_at   TIMESTAMP,  
  updated_at   TIMESTAMP  
);
```

If orchestrator crashes: restart, read saga state from DB, resume from last step

Part B — Advanced Data Patterns

Chapter 5: CQRS

Q7 ■ ■ What is CQRS? When is it worth the complexity?

Plain English First

CQRS = **Command Query Responsibility Segregation**.

In traditional architecture, the same data model handles both reads and writes. With CQRS, you have **separate models** for reading (queries) and writing (commands). Like a restaurant where one team takes orders (commands) and a different team serves food (queries) — each optimized for their role.

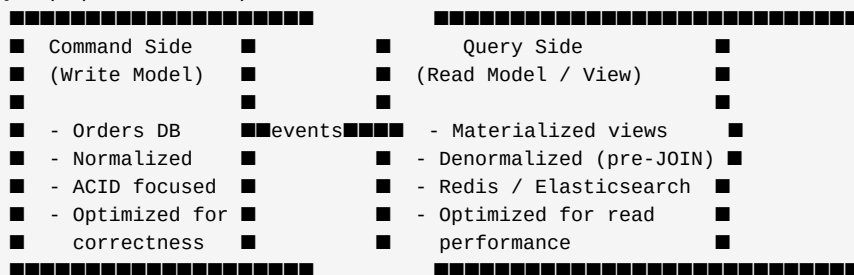
Traditional (single model):

Write: UPDATE orders SET status='SHIPPED' WHERE id=123

Read: SELECT o.*, u.name, p.title FROM orders o JOIN users u JOIN products p ...

Same DB, same schema – hard to optimize both at once

CQRS (separate models):



Full Example — Order Dashboard

Command side (write – PostgreSQL):

Normalized, ACID, handles business rules

TABLE orders (id, user_id, status, created_at)

TABLE order_items (order_id, product_id, quantity, price)

On "ShipOrder" command:

1. Validate order exists and is in PENDING state
2. UPDATE orders SET status='SHIPPED'
3. Publish OrderShipped event to Kafka

Query side (read – Elasticsearch or Redis):

Pre-computed, denormalized view of what the dashboard needs

```
{
  "orderId": 123,
  "customerName": "Alice",
  "customerEmail": "alice@example.com",
  "status": "SHIPPED",
  "items": [{"product": "iPhone 15", "qty": 1, "price": 999}],
  "totalAmount": 999,
  "shippedAt": "2026-05-07T10:00:00Z"
}
```

Event Consumer (syncs write → read model):

Listens to OrderShipped event

Updates the Elasticsearch document (or Redis hash) for order 123

Read API query: GET /orders/123 → Elasticsearch (no join needed, instant)

When CQRS is worth it:

- ✓ Read and write workloads have very different shapes (complex reads, simple writes)
- ✓ Need different scaling for reads vs writes (100:1 read/write ratio)
- ✓ Different latency requirements (writes can be async, reads must be fast)
- ✓ Multiple read views of the same data (dashboard, mobile app, reporting)

When NOT to use CQRS:

- ✗ Simple CRUD with no complex read requirements
- ✗ Small teams (operational overhead is significant)
- ✗ Eventual consistency is unacceptable (CQRS read models lag behind writes)

Chapter 6: Event Sourcing

Q8 ■ ■ What is Event Sourcing? How is it different from CRUD?

Plain English First

Traditional CRUD stores only the **current state**:

```
Account balance: $1,500
```

Event Sourcing stores every **event that ever happened**:

```
AccountOpened($0) → Deposited($2000) → Withdrew($300) → Deposited($500) → Withdrew($700)
Replay → $2000 - $300 + $500 - $700 = $1,500
```

Like double-entry bookkeeping — you never cross out the old amount, you always add a new line.

CRUD approach:

```
CREATE: INSERT INTO accounts VALUES (1, 1500)
UPDATE: UPDATE accounts SET balance = 1200 WHERE id = 1
← old state is GONE — you can't tell what happened or why
```

Event Sourcing:

```
Event 1: { type: "AccountOpened",   accountId: 1, initialBalance: 0,   timestamp: t1 }
Event 2: { type: "MoneyDeposited",   accountId: 1, amount: 2000,       timestamp: t2 }
Event 3: { type: "MoneyWithdrawn",   accountId: 1, amount: 300,         timestamp: t3 }
Event 4: { type: "MoneyDeposited",   accountId: 1, amount: 500,         timestamp: t4 }
Event 5: { type: "MoneyWithdrawn",   accountId: 1, amount: 700,         timestamp: t5 }
```

Current state = replay all events:

```
balance = 0 + 2000 - 300 + 500 - 700 = 1500
```

What you get for FREE:

- ✓ Complete audit log — regulators love this
- ✓ Time travel — "what was the balance on March 1st?" (replay up to that date)
- ✓ Bug forensics — "how did balance become negative?" (replay and find the bug)
- ✓ Event-driven integration — events are publishable to Kafka, other services
- ✓ Retroactive projections — build a NEW view of old data (add a feature, replay all history)

Implementation

```

// Event Store (append-only)
CREATE TABLE account_events (
    event_id      BIGSERIAL PRIMARY KEY,
    account_id    BIGINT NOT NULL,
    event_type    VARCHAR(50) NOT NULL,
    payload       JSONB NOT NULL,
    occurred_at   TIMESTAMP NOT NULL DEFAULT NOW(),
    version       INT NOT NULL    -- optimistic concurrency: expected version on write
);

// Aggregate – rebuilt by replaying events
public class BankAccount {
    private Long id;
    private BigDecimal balance;
    private int version;
    private List<DomainEvent> uncommittedEvents = new ArrayList<>();

    // Reconstruct from event history
    public static BankAccount reconstitute(List<AccountEvent> history) {
        BankAccount account = new BankAccount();
        history.forEach(account::apply);
        return account;
    }

    public void deposit(BigDecimal amount) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Amount must be positive");

        // Record the event – don't change state directly
        MoneyDeposited event = new MoneyDeposited(id, amount, LocalDateTime.now());
        apply(event);           // Apply to this instance
        uncommittedEvents.add(event); // Queue for persistence
    }

    private void apply(MoneyDeposited event) {
        this.balance = this.balance.add(event.getAmount());
        this.version++;
    }

    // Similarly: apply(MoneyWithdrawn), apply(AccountOpened)...
}

// Snapshots – performance optimization for long event histories
// After 100 events: save current state as snapshot
// Replay = load snapshot + events after snapshot (not all 10,000 events)
CREATE TABLE account_snapshots (
    account_id    BIGINT PRIMARY KEY,
    balance       NUMERIC,
    version       INT,           -- snapshot taken at this version
    created_at    TIMESTAMP
);

```

When to use Event Sourcing

- ✓ Audit requirement (banking, healthcare, legal – must explain every state change)
 - ✓ Complex business logic with multiple state transitions
 - ✓ Need time-travel / historical queries
 - ✓ Microservices that need to integrate via events

 - ✗ Simple CRUD (adds significant complexity)
 - ✗ Teams unfamiliar with the pattern (steep learning curve)
 - ✗ High-frequency updates to same aggregate (event log grows huge, snapshots essential)
-

Chapter 7: Data Pipelines

Q9 ■ ■ What is the Lambda Architecture? What is the Kappa Architecture?

Lambda Architecture

Problem: you need both real-time results AND accurate historical results

Real-time: show live sales dashboard (approximate, fast)

Batch: run accurate end-of-day reports (exact, slow)

Lambda = Two parallel paths:

Raw Data ■■■■■ Batch Layer (Hadoop/Spark) ■■ Batch Views ■■
■ Processes ALL historical data (accurate) ■■■■ Serving Layer ■■■ Query
■■■■ Speed Layer (Kafka Streams) ■■ Real-time ■■
Processes recent data only Views
(last few hours) (fast, approx.)

Batch Layer: Spark jobs run every hour/day

- Reads ALL data from S3/HDFS
- Computes exact results
- Overwrites batch views in DB

Speed Layer: Kafka Streams / Flink

- Processes only new data (since last batch)
- Updates real-time views (Redis/Cassandra)
- Results are approximate (missing late-arriving data)

Query time: merge batch view + speed view → complete answer

Problems with Lambda:

- Two codebases to maintain (batch + streaming) – same logic implemented twice
- Operational complexity (Hadoop + Kafka + serving layer)
- Batch results lag by hours

Kappa Architecture

Insight: if you can replay Kafka from the beginning, you don't need a separate batch layer!

Raw Data ■■■ Kafka (immutable log) ■■■ Streaming Job (Flink/Kafka Streams) ■■■ Serving Layer

Re-processing (when you fix a bug or add a feature):

- Start a NEW streaming job that reads Kafka from offset 0 (beginning)

- Write results to a NEW output table

- When new job catches up to current time → swap serving layer to new output

- Kill old job + old output table

Pros: Single codebase, simpler to operate

Cons: Kafka must retain ALL data (expensive for years of history)

- Replay can be slow for large historical datasets

When to use each:

- Lambda: Data older than Kafka retention needed, different tech for batch vs stream

- Kappa: Team can afford long Kafka retention, prefers single codebase

- Modern trend: Kappa with Apache Flink + S3 as long-term storage

Part C — Reliability & Observability

Chapter 8: Observability

Q10 ■ ■ What is the difference between Monitoring and Observability? Explain the three pillars.

Monitoring: checking known failure modes ("is CPU > 80%?")

You know what to check in advance

Good for: known-unknowns

Observability: ability to understand ANY internal state from external outputs

You can debug unknown failures without pre-defining checks

Good for: unknown-unknowns (novel failures you've never seen before)

Three Pillars:

■ Metrics	■ Numeric aggregates over time	■
■	■ "HTTP errors per second in the last 5 minutes"	■
■	■ Tools: Prometheus + Grafana, Micrometer, Datadog	■
■	■ Types: Counter, Gauge, Histogram, Summary	■

■ Traces	■ End-to-end journey of a single request across services	■
■	■ "This request took 450ms: 5ms in API Gateway, 200ms in	■
■	■ User Service, 240ms in DB query on users table"	■
■	■ Tools: Jaeger, Zipkin, AWS X-Ray, OpenTelemetry	■
■	■ Concepts: Trace (full journey), Span (one operation)	■

■ Logs	■ Discrete events with context	■
■	■ "2026-05-07 ORDER-123 FAILED: inventory unavailable"	■
■	■ Tools: ELK Stack (Elasticsearch + Logstash + Kibana),	■
■	■ Loki + Grafana, CloudWatch	■
■	■ Best practice: structured JSON logs, correlation ID	■

Distributed Tracing in Practice

Every request gets a trace-id at the entry point (API Gateway)
Propagated via HTTP headers: X-Trace-Id: abc123, X-Span-Id: def456

Microservice A receives request:

Creates span: {traceId: abc123, spanId: s1, service: "order-svc", operation: "placeOrder"}
Calls Microservice B with same trace-id, new span-id

Microservice B:

Creates child span: {traceId: abc123, spanId: s2, parentSpanId: s1, service: "inventory-svc"}
Calls DB:
Creates child span: {traceId: abc123, spanId: s3, parentSpanId: s2, operation: "SELECT..."}

Trace visualization (Jaeger / Zipkin):

abc123 ■ Total: 250ms
■■■ order-svc.placeOrder (250ms)
■ ■■■ inventory-svc.checkStock (100ms)
■ ■ ■■■ DB query: SELECT (95ms) ← BOTTLENECK
■ ■■■ payment-svc.charge (80ms)
■■■ notification-svc.send (30ms, async)

Without tracing: "the API is slow, but I don't know where"

With tracing: "the inventory DB query takes 95ms, needs an index"

Key Metrics to Track (RED Method)

RED Method (for every service):

R – Rate: requests per second
E – Errors: error rate (% of requests returning 5xx)
D – Duration: P50, P95, P99 latency

USE Method (for every resource – CPU, memory, disk, network):

U – Utilization: what % of resource is being used
S – Saturation: how much is queued (waiting)
E – Errors: error count

Dashboard must-haves at Apple scale:

- Request rate (RPS) per endpoint
- Error rate with alert at > 0.1%
- P99 latency with alert at SLO threshold
- DB connection pool: active, idle, pending
- Cache hit rate (should be > 95% for hot paths)
- Queue depth (if using Kafka – consumer lag)
- JVM heap usage + GC pause duration

Chapter 9: Global Distribution & Multi-Region

Q11 ■ ■ How do you design a multi-region system? What are the key trade-offs?

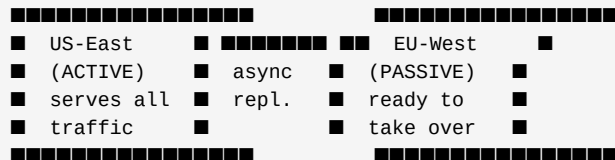
Why multi-region:

Latency: users in Tokyo get < 10ms if served from Tokyo region vs 200ms from US
Availability: if US-East datacenter burns down, EU and APAC still serve users
Data residency: GDPR requires EU user data stays in EU (legal mandate)

Deployment patterns:

1. Active-Passive (Warm Standby)

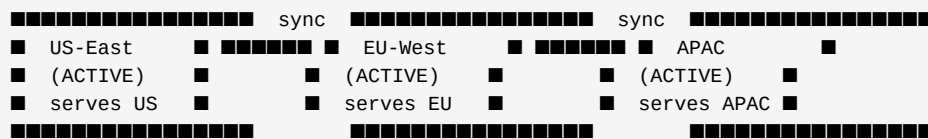
Primary region handles ALL traffic
Secondary region is a warm replica (data synced, but not serving traffic)
On primary failure: DNS failover to secondary (takes 30-120 seconds)



Pros: Simple, no write conflict issues
Cons: Users in EU get high latency (hits US), secondary wastes resources

2. Active-Active (Multi-master)

All regions serve reads AND writes
Data synchronized bi-directionally



Pros: Local reads + writes for all users, full HA
Cons: Write conflicts (two users update same record in different regions simultaneously)
Hard to achieve strong consistency (speed of light limits cross-region sync)

3. Geo-Partitioned (Data Sharding by Region)

EU users' data ONLY lives in EU region
US users' data ONLY lives in US region
No cross-region sync needed for most operations

Perfect for GDPR compliance
Problem: what if an EU user travels to US and writes data? Route back to EU

Write Conflict Resolution (Active-Active):

Last Write Wins (timestamp): simplest, risk of data loss if clocks skew
CRDT: data types that automatically merge (sets, counters)
Custom merge function: application defines how to resolve conflicts
Avoid by partitioning: user X always writes to region X (sticky routing)

Global Load Balancing

Anycast DNS (Cloudflare, AWS Route 53):

- Single IP address, multiple servers worldwide

- Router automatically sends traffic to geographically nearest server

- Client sends to 1.1.1.1 → routed to nearest Cloudflare PoP

GeoDNS:

- DNS returns different IPs based on client's location

- EU client → DNS returns EU server IPs

- US client → DNS returns US server IPs

Health-check based failover:

- Route 53 health checks each region

- US-East unhealthy → stop routing US traffic there → redirect to US-West

- Failover takes 30-60 seconds (DNS TTL)

Chapter 10: Real-Time Systems

Q12 ■ ■ WebSockets vs Server-Sent Events vs Long Polling — when to use each?

HTTP Short Polling (baseline):

Client asks every N seconds: "anything new?"

Server responds immediately (empty if nothing new)

Pros: Simple, stateless

Cons: High server load (empty responses), poor latency (up to N seconds delay)

Use: Last resort, legacy clients

Long Polling:

Client sends request, server HOLDS it open until there's data (or timeout)

Server sends data → client immediately sends next long-poll request

Pros: Near-real-time, works everywhere

Cons: Ties up HTTP connections, overhead per message

Use: Notification systems, simple real-time (when WebSocket is overkill)

Server-Sent Events (SSE):

Client opens ONE HTTP connection, server pushes events continuously

One-directional: server → client only

Automatic reconnection built into browser

Pros: Simple, works over HTTP/1.1, built-in reconnect, EventSource API in browser

Cons: Unidirectional (can't send from client), limited to text

Use: Live feeds (scores, stock tickers, news), progress updates, notifications

WebSockets:

Full-duplex: client ↔ server both can send anytime

Single TCP connection, low overhead per message (no HTTP headers)

Pros: True bidirectional, low latency, efficient (frames instead of full HTTP)

Cons: Stateful (sticky sessions or connection registry), harder to scale, no HTTP caching

Use: Chat, live collaboration (Google Docs), multiplayer games, trading platforms

Decision tree:

Client needs to SEND messages? → WebSocket

Server just pushes updates? → SSE (simpler, auto-reconnect)

Legacy browser/proxy issues? → Long Polling

Simple polling is fine (analytics dashboard)? → Short Polling

WebSocket Scaling

Problem: WebSockets are stateful – user A's connection is on Server 1

If you have 4 servers and 1M connections:

Each server holds ~250K connections

Message to user A MUST go to Server 1

Solution: Pub/Sub via Redis

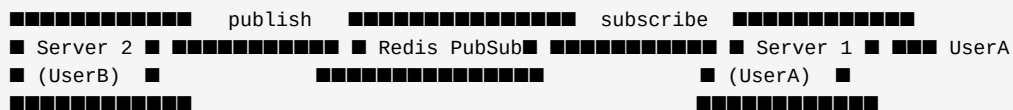
Server 1: User A connected here

Server 2: User B connected here

User B sends message to User A:

Server 2 → can't reach A directly → publishes to Redis channel "user:A"

Server 1 → subscribes to "user:A" → receives message → pushes to User A's WebSocket



At massive scale: replace Redis with Kafka

Each server is a Kafka consumer group member

Message published to topic → all servers receive → each delivers to connected users

Q13 ■ What is CRDT? How does it enable conflict-free collaboration?

CRDT = Conflict-free Replicated Data Type

A data type designed so that concurrent updates from multiple users/nodes can always be merged automatically and deterministically – no conflicts ever.

The math: operations must be commutative, associative, and idempotent

Commutative: A then B = B then A (order doesn't matter)

Associative: $(A+B)+C = A+(B+C)$

Idempotent: $A+A = A$ (applying same update twice has no extra effect)

CRDT Types:

G-Counter (Grow-only counter – only increments):

Each node maintains its own counter vector

Node1: [5, 0, 0] (node1=5, node2=0, node3=0)

Node2: [0, 3, 0] (node1=0, node2=3, node3=0)

Merge: take max of each position $\rightarrow [5, 3, 0] = \text{total } 8$

No conflicts possible!

LWW-Register (Last Write Wins Register):

Each update tagged with a timestamp

Merge: higher timestamp wins

Risk: clock skew can cause updates to be lost

OR-Set (Observed-Remove Set):

Each element tagged with a unique ID when added

Remove = mark that unique ID as removed

Merge: element present if any add-ID not marked removed

Handles: concurrent add + remove of same element

Text CRDT (used in Google Docs, Figma):

Each character assigned unique, immutable position ID (e.g., "between pos3 and pos4")

Insert: "hello" $\rightarrow H(\text{pos1}), e(\text{pos1.5}), l(\text{pos1.75}), l(\text{pos1.875}), o(\text{pos1.9375})$

Concurrent inserts at same position: order by node ID (deterministic)

Delete: mark character as tombstone (don't actually remove – position IDs are permanent)

Libraries: Yjs, Automerge (production CRDT implementations)

Real-world use:

Google Docs: collaborative text editing

Figma: collaborative design canvas

Notion: collaborative documents

Redis: CRDT-based multi-master (Redis Enterprise CRDT)

Part D — Senior-Level Design Problems

Chapter 11: Design Google Docs

Q14 ■ ■ Design a real-time collaborative document editor like Google Docs

Step 1: Requirements

Functional:

- Multiple users edit the same document simultaneously
- Changes appear in real-time for all collaborators (< 100ms)
- Conflict resolution when users type at the same time
- Cursor positions of other users visible
- Document versioning / undo-redo
- Offline editing with sync on reconnect

Scale:

- 1B documents stored
- 10M concurrent active editors
- Average document: 100KB
- Peak: viral documents with 10,000 simultaneous editors

Step 2: Core Algorithm — Operational Transformation vs CRDT

Two approaches to conflict resolution:

Operational Transformation (OT) – Google's original approach:

User A inserts "X" at position 5

User B simultaneously deletes character at position 5

Server applies A's op first, then transforms B's op: delete at position 6 (shifted by insert)

Pro: well-understood, efficient

Con: complex to implement correctly (Google has hundreds of test cases), requires server coordination

CRDT – modern approach (used by Figma, Notion):

Each character gets a permanent unique position ID (fractional indexing)

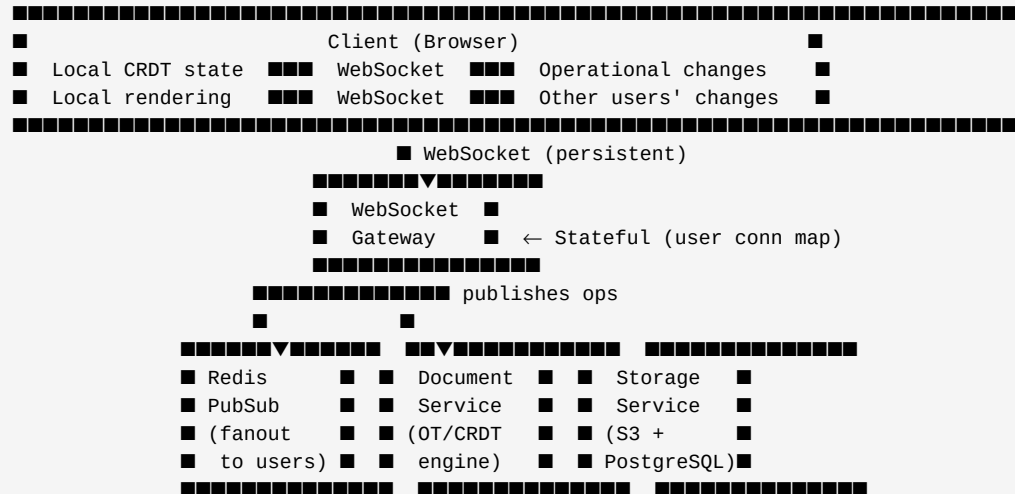
No transformation needed – merge is always deterministic

Works offline natively (merge on reconnect)

Pro: peer-to-peer capable, offline-first, mathematically correct

Con: tombstones accumulate (deleted chars stay as placeholders), more memory

Step 3: Architecture



Step 4: Operation Flow

User A types "Hello" at position 3:

1. Client A applies operation locally (optimistic) → instant visual feedback
2. Client A sends to WebSocket Gateway:

```
{ docId: "doc123", userId: "A", op: INSERT, pos: 3, char: "H", version: 42 }
```
3. Document Service receives op:
 - a. Acquires distributed lock on doc123 (Redis SETNX)
 - b. Validates: is version 42 current? (prevent stale ops)
 - c. Transforms op against concurrent ops (if any)
 - d. Applies to document state
 - e. Stores op in Ops Log (PostgreSQL – append only)
 - f. Publishes to Redis channel "doc:doc123"
 - g. Releases lock
4. WebSocket Gateway is subscribed to "doc:doc123":
 Pushes transformed op to ALL other connected users (User B, C, D...)
5. Client B, C, D apply the op to their local state:

```
{ docId: "doc123", userId: "A", op: INSERT, pos: 3, char: "H", version: 43 }
```

Cursor sharing: WebSocket messages (not stored, ephemeral)

```
{ type: "cursor", userId: "B", position: 15, color: "#ff5733" }
```


 Fanout to all other users in same doc session

Step 5: Storage

```

-- Document metadata
CREATE TABLE documents (
  doc_id      UUID PRIMARY KEY,
  title       TEXT,
  owner_id    BIGINT,
  created_at  TIMESTAMP,
  version     BIGINT DEFAULT 0
);

-- Operation log (append-only, source of truth for document state)
CREATE TABLE document_ops (
  op_id       BIGSERIAL PRIMARY KEY,
  doc_id      UUID NOT NULL,
  user_id     BIGINT,
  op_type     VARCHAR(10), -- INSERT, DELETE, RETAIN
  position    INT,
  content     TEXT,
  version     BIGINT,      -- doc version BEFORE this op
  applied_at  TIMESTAMP DEFAULT NOW()
);

-- Snapshots (reconstruct doc without replaying all ops)
-- Every 1000 ops: serialize full document state to S3
-- On load: fetch latest snapshot + ops since snapshot

```

Chapter 12: Design a Distributed Cache

Q15 ■ ■ Design a distributed in-memory cache like Redis

Step 1: Core Requirements

Functional:

- GET key → value
- SET key value [TTL]
- DEL key
- Support TTL (expiration)
- Pub/Sub messaging
- Persistence (optional: survive restarts)

Non-functional:

- < 1ms P99 latency for GET/SET
- 10M operations/second per cluster
- 99.999% availability
- Horizontal scalability

Step 2: Single Node Design

Data structures (Redis-style):

Strings: key → byte array (any value)
Hash: key → {field: value, ...} (JSON-like, field-level access)
List: key → [a, b, c, ...] (queue / timeline)
Set: key → {a, b, c} (unique) (tags, memberships)
Sorted Set: key → {a:1.0, b:2.0} (leaderboards, rate limiting)

Memory management:

All data in RAM (primary)
Hash table internally: key → pointer to value object
LRU eviction when memory full (configurable policy)

Persistence options:

RDB (snapshot): write full dataset to disk every N seconds
Fast restart, some data loss (up to N seconds)
AOF (append-only file): log every write command
Slower, but near-zero data loss (can replay on restart)
Hybrid: RDB + AOF (Redis default in production)

Single-threaded I/O:

Redis processes commands in a SINGLE thread (no lock contention)
Achieves 100K+ ops/sec per core – no mutex overhead
I/O multiplexing (epoll/kqueue): one thread handles thousands of connections

Step 3: Distributed Cluster Design

Sharding with Consistent Hashing:

16384 hash slots total

```
cluster assigns slots to nodes
```

Node1: slots 0-5460

Node2: slots 5461-10922

Node3: slots 10923-16383

```
Hash: slot = CRC16(key) % 16384
```

Client knows slot $\rightarrow$ node mapping (cluster map)

Client routes directly to correct node (no coordinator needed!)

Adding a node:

Assign some slots to new node

```
Migrate data for those slots (live, no downtime)
```

Only $\sim 1/N$ of keys move

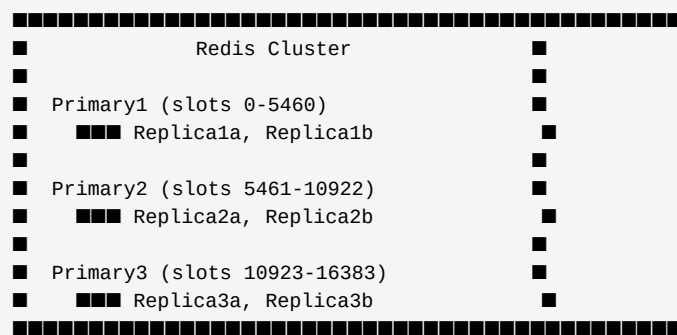
Replication:

Each primary node has N replicas

Writes → primary only (replicated async)

Reads → primary or replica (configurable)

Primary failure → replica promoted (automatic with sentinel or cluster mode)



Chapter 13: Design a Payment System

Q16 ■ ■ Design a payment system like Apple Pay

Step 1: Requirements

Functional:

- Initiate payment (charge user's card/bank)
- Receive payment (merchant receives funds)
- Refund a payment
- View payment history

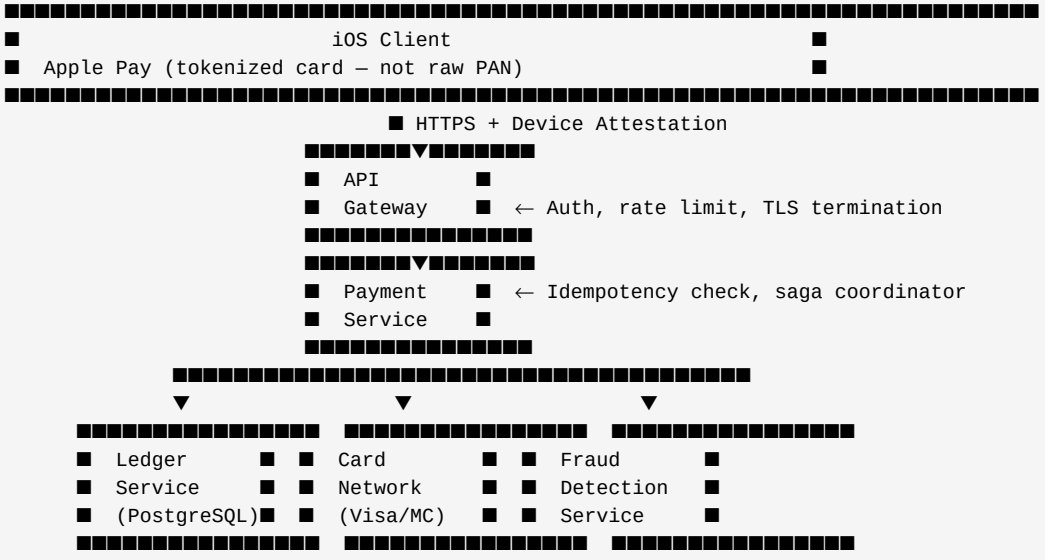
Non-functional:

- Exactly-once processing (NEVER double-charge)
- 99.999% availability (downtime = revenue loss)
- P99 latency < 3 seconds (includes card network round-trip)
- Audit trail for every state change (compliance)
- PCI-DSS compliance (never store raw card numbers)

Step 2: Key Design Principles

1. Idempotency (most critical)
Client generates idempotency-key (UUID) per payment
Server stores: idempotency_key → payment_id, result
Retry with same key → return cached result, skip processing
Guarantee: no matter how many retries, exactly one charge
2. State machine – payment has well-defined states
INITIATED → PENDING → PROCESSING → SUCCEEDED / FAILED / REFUNDED
3. Saga pattern for distributed steps
 1. Debit user's wallet / initiate card charge
 2. Call card network (Visa/Mastercard API)
 3. Credit merchant account
 4. Send receiptsEach step has a compensating action (refund)
4. Double-entry bookkeeping
Every transaction = at least two ledger entries
Debit user account + Credit merchant account
Sum of all ledger entries always = 0 (conservation of money)

Step 3: Architecture



Step 4: Database Schema (Ledger)

```

-- Payments table (saga state)
CREATE TABLE payments (
  payment_id      UUID PRIMARY KEY,
  idempotency_key VARCHAR(255) UNIQUE NOT NULL, -- client-provided, prevent duplicates
  payer_id        BIGINT NOT NULL,
  merchant_id     BIGINT NOT NULL,
  amount          NUMERIC(19,4) NOT NULL,
  currency        CHAR(3) NOT NULL,
  status          VARCHAR(20) NOT NULL, -- INITIATED, PENDING, SUCCEEDED, FAILED
  card_token      VARCHAR(255), -- tokenized (never raw PAN)
  network_txn_id  VARCHAR(255), -- Visa/MC transaction reference
  created_at      TIMESTAMP DEFAULT NOW(),
  updated_at      TIMESTAMP,
  version         INT DEFAULT 0 -- optimistic locking
);

-- Ledger (double-entry, append-only, NEVER update)
CREATE TABLE ledger_entries (
  entry_id        BIGSERIAL PRIMARY KEY,
  payment_id      UUID NOT NULL,
  account_id      BIGINT NOT NULL,
  entry_type      VARCHAR(10) NOT NULL, -- DEBIT or CREDIT
  amount          NUMERIC(19,4) NOT NULL, -- always positive
  currency        CHAR(3) NOT NULL,
  created_at      TIMESTAMP DEFAULT NOW()
);

-- For payment of $99.99:
-- DEBIT user_account $99.99
-- CREDIT merchant_account $96.99 (after 3% fee)
-- CREDIT platform_account $3.00 (fee)
-- Sum of all = 0 ✓

-- Accounts table
CREATE TABLE accounts (
  account_id      BIGINT PRIMARY KEY,
  owner_id        BIGINT NOT NULL,
  account_type    VARCHAR(20), -- USER, MERCHANT, PLATFORM
  currency        CHAR(3)
);

-- Balance = computed from ledger (not stored separately)
-- SELECT SUM(CASE WHEN entry_type='CREDIT' THEN amount ELSE -amount END)
-- FROM ledger_entries WHERE account_id = ?
-- Slower but always accurate (no sync issues)

```

Step 5: Handling Partial Failures

Scenario: Payment Service calls Visa, Visa charges card, then Payment Service crashes before recording the result in the DB.
On recovery: did the charge go through?

Solution: Query Visa using the idempotency_key to check payment status
Payment Service → GET /visa/payments?idempotency_key=abc123
If Visa: "payment charged successfully" → record as SUCCEEDED, don't charge again
If Visa: "no payment found" → retry the charge
If Visa: "payment failed" → mark as FAILED

Timeout handling:
Visa response takes > 30s → Payment Service times out
State = PENDING (unknown)
Background job: find PENDING payments > 5 min → query network for status → resolve

Chapter 14: Design Uber / Lyft

Q17 ■ ■ Design a real-time ride-sharing system like Uber

Step 1: Requirements

Functional:

- Rider requests a ride (origin + destination)
- System matches rider to nearest available driver
- Driver location tracked in real-time (GPS every 5s)
- ETA shown to rider
- Surge pricing based on demand/supply

Scale:

- 20M rides per day → ~230 rides/sec
- 5M active drivers at peak → 5M GPS updates every 5s = 1M writes/sec
- Matching must complete in < 5 seconds
- Location accuracy: 10-meter precision

Step 2: Location Storage — The Core Challenge

Problem: How do you efficiently find all drivers within x km of a point?

Naive approach:

```
SELECT * FROM drivers WHERE
  lat BETWEEN (rider_lat - 0.1) AND (rider_lat + 0.1) AND
  lng BETWEEN (rider_lng - 0.1) AND (rider_lng + 0.1);
→ Full table scan, 5M rows → too slow
```

Solution 1: Geohashing

Divide Earth into grid cells, each with a string code
Precision: 6-char geohash = ~1.2km × 0.6km cell
7-char geohash = ~153m × 153m cell

Driver at (37.7749, -122.4194) → geohash "9q8yy"

Store: Redis HSET "geo:9q8yy" driver_id {lat, lng, status}

Find nearby drivers:

1. Compute rider's geohash: "9q8yy"
2. Query rider's cell + 8 neighboring cells (3×3 grid)
3. Filter results by actual distance

9 Redis lookups → O(1) per cell → very fast

Solution 2: Redis Geo Commands (built-in)

```
GEOADD drivers:active {lng} {lat} {driver_id}
GEORADIUS drivers:active {rider_lng} {rider_lat} 3 km WITHCOORD COUNT 20
→ Returns up to 20 drivers within 3km, with coordinates
```

Redis Sorted Set internally using geohash score

O(N+log(M)) where N=results, M=total drivers → fast for sparse results

Step 3: Architecture

```
Driver App → [GPS Ingestion API] → Kafka → [Location Consumer] → Redis Geo
```

Consumer batch-writes to Redis

Driver table in PostgreSQL: status, last_seen, trip_id (durable state)

Matching Flow:

Matching Service:

- ```

■ Matching Service ■
■ ■
■ Redis Geo → candidates → filter → rank → send offers ■
■ ■
■ Timeout: driver has 10s to accept, else try next ■

```

Driver sends GPS every 5s → published to trip-specific Kafka partition

Rider app subscribes via WebSocket (pushed every 5s)

ETA recalculated using routing engine (OSRM / Google Maps API)

```
Supply: # available drivers in area
Demand: # ride requests in area (last 5 minutes)
```

```
surge_multiplier = max(1.0, f(demand / supply))
demand/supply < 0.5 → 1.0x (plenty of drivers)
demand/supply = 1.0 → 1.5x
demand/supply = 2.0 → 2.5x
```

Computed per geohash cell, updated every minute

Stored in Redis: GET "surge:9q8yy" → "1.8"

How to compute:

Kafka Streams aggregates: count ride requests + count available drivers

Per geohash cell, per 1-minute tumbling window

Output: surge multiplier  $\rightarrow$  written back to Redis

# Chapter 15: Design a Distributed Search Engine

---

## Q18 ■ ■ Design a web-scale search engine (or App Store search)

---

### Step 1: Requirements

Scope: App Store search (simpler and more relevant than web search for Apple)

Functional:

- Index: app name, description, category, developer, screenshots captions
- Search: keyword query → ranked list of apps
- Ranking factors: relevance (text match), popularity (downloads, ratings), freshness
- Faceted filtering: category, price, rating  $\geq 4.0$
- Autocomplete / typeahead

Scale:

- 2M apps indexed
- 1B searches/day → ~11,500 searches/sec
- Search latency: < 100ms P99
- Index update: app update visible in < 5 minutes

### Step 2: Core Concepts — Inverted Index

Forward index (document → words):

App123: ["photo", "editor", "filter", "retro", "vintage"]

App456: ["photo", "share", "social", "camera"]

Inverted index (word → documents):

"photo" → [App123, App456] ← all apps containing "photo"

"editor" → [App123]

"filter" → [App123]

"share" → [App456]

Query "photo editor":

"photo" matches: [App123, App456]

"editor" matches: [App123]

Intersect: [App123] → "photo editor" is App123

With TF-IDF scoring:

TF (Term Frequency): how often "photo" appears in App123's description

IDF (Inverse Doc Frequency): how rare is "photo" across all apps (less rare = less valuable)

Score = TF × IDF → rank by score descending

Modern approach: BM25 (used by Elasticsearch, Lucene)

Improves on TF-IDF: diminishing returns for term repetition, normalizes for doc length

### Step 3: Architecture

#### Indexing Pipeline:

App Update → Kafka ("app-updates") → Index Worker → Elasticsearch Cluster

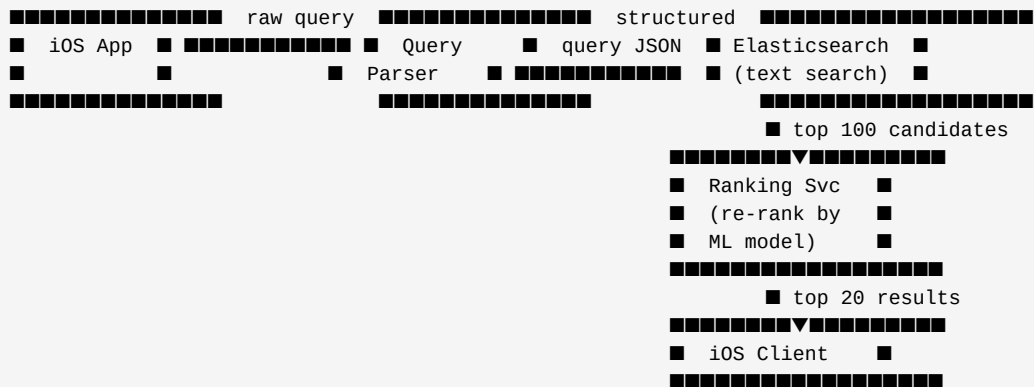


#### Index Worker:

1. Fetch full app document (description, screenshots, metadata)
2. Preprocess: lowercase, remove stopwords, stem ("editing" → "edit")
3. Compute field weights: title match worth 10×, description 1×
4. PUT /apps/\_doc/{app\_id} → Elasticsearch

#### Search Pipeline:

Client → [Search API] → [Query Parser] → Elasticsearch → [Ranking Service] → Response



### Step 4: Elasticsearch Cluster Design

Index: "apps" (2M documents)  
Shards: 10 primary shards (split data evenly)  
Replicas: 1 replica per shard (20 shards total, HA + read scaling)

Shard assignment (6-node cluster):  
Node1: P0, P1, R4, R5  
Node2: P2, P3, R0, R1  
Node3: P4, P5, R2, R3  
(P=primary, R=replica – replica on different node than its primary)

Query routing:  
Search request hits any node (coordinator)  
Coordinator fans out to all 10 primary shards (or replicas)  
Each shard searches its portion of data  
Coordinator merges results, sorts, returns top 20

Performance optimizations:  
- Shard size: aim for 10-50GB per shard (sweet spot for Elasticsearch)  
- Warm replicas: pre-load hot shards in memory (shard allocation awareness)  
- Query cache: cache frequent queries (LRU, invalidated on index update)  
- Doc values: pre-compute sort fields for fast sorting without loading all fields

## Step 5: Autocomplete

Problem: suggest completions as user types "ph" → ["photo editor", "photos", "phone"]

Solution: Trie or Completion Suggester

Elasticsearch Completion Suggester:  
Store suggestions in special "completion" field type  
Backed by FST (Finite State Transducer) in memory – very fast prefix matching  
Query: GET /apps/\_search/suggest { "text": "ph", "completion": { "field": "suggest" } }  
Returns: ["photo editor", "photos", "phone"] in < 10ms

For higher QPS (100K+ typeahead/sec):  
Pre-compute top-K suggestions for every prefix  
Store in Redis: GET "suggest:ph" → ["photo editor", "photos", "phone"]  
Refreshed hourly (or on popularity change)  
Sub-millisecond response, no Elasticsearch hit

Ranking autocomplete suggestions:  
Sort by: search\_frequency × recency\_weight × app\_quality\_score  
"photo editor" searched 10M times/day → ranked above "photo effects" (1M/day)

---

## Senior Staff Engineer Quick Reference

### Distributed Systems Decision Matrix

| Scenario                  | Solution                     | Why                          |
|---------------------------|------------------------------|------------------------------|
| Leader election           | Raft (etcd/ZooKeeper)        | Linearizable, fault-tolerant |
| Distributed lock          | Redis SETNX + TTL            | Simple, fast, auto-expire    |
| Distributed counter       | CRDT G-Counter or Redis INCR | Conflict-free, atomic        |
| Cross-service transaction | Saga pattern                 | No 2PC overhead, async       |
| Audit trail               | Event Sourcing               | Immutable history            |
| Read/write separation     | CQRS                         | Independent scaling          |
| Real-time collaboration   | CRDT (Yjs/Automerger)        | Conflict-free merges         |
| Geo-spatial search        | Redis Geo / Geohash          | O(1) radius queries          |
| Full-text search          | Elasticsearch                | Inverted index, BM25         |
| Multi-region writes       | Active-Active + CRDT         | Highest availability         |
| Stream processing         | Apache Flink / Kafka Streams | Stateful, exactly-once       |

## Back-of-Envelope for Senior Interviews

### Data sizes:

Twitter: 500M tweets/day × 140 bytes = 70 GB/day → 25 TB/year  
Uber: 5M drivers × GPS every 5s = 1M writes/sec  
Google: 8.5B searches/day = 100K searches/sec

### Estimation shortcuts:

1 million req/day = ~12 req/sec  
1 billion req/day = ~12,000 req/sec  
1 TB data =  $10^{12}$  bytes  
1 PB data =  $10^{15}$  bytes  
PostgreSQL: ~10K writes/sec (single primary, indexed)  
Cassandra: ~100K writes/sec (per node, wide-column)  
Redis: ~1M ops/sec (single node, in-memory)  
Kafka: ~10M msgs/sec (per cluster)

### Replication factor:

Always use 3 replicas: tolerates 1 failure, quorum = 2  
"I'd deploy with replication factor 3 across 3 AZs for 99.99% availability"

### Consensus quorum:

3 nodes → quorum = 2 → tolerates 1 failure  
5 nodes → quorum = 3 → tolerates 2 failures  
7 nodes → quorum = 4 → tolerates 3 failures



>

Key themes Apple Senior Staff Engineer interviews probe: - **Theoretical foundations:** CAP/PACELC, consistency models, Raft — not just surface level - **Correctness under failure:** idempotency, saga compensation, exactly-once semantics - **Quantified trade-offs:** "this gives us X but costs Y — I'd choose it because..." - **Global scale thinking:** multi-region design, data residency, conflict resolution - **Cross-system coherence:** how CQRS + Event Sourcing + Saga fit together - **Algorithm intuition:** why consistent hashing, why inverted index, why CRDT